DESIGN AND APPLY CONTEXT-AWARE ACCESS CONTROL BASED ON USER IDENTITY

INTRODUCTION

In modern digital systems, protecting sensitive information and resources is a critical requirement. Traditional access control mechanisms mainly rely on user roles or predefined permissions, which may not be sufficient to handle today's dynamic security challenges. Context-aware access control enhances security by considering not only the user's identity but also additional contextual factors such as location, device type, time of access, and network conditions. By integrating these contextual attributes with identity-based authentication, organizations can create more adaptive and intelligent access control systems. Designing and applying context-aware access control based on user identity helps ensure that only authorized users can access specific resources under appropriate conditions, thereby improving security, reducing risks, and supporting modern cybersecurity frameworks.

Securing systems and sensitive data has become increasingly important. Traditional access control mechanisms mainly rely on user roles or fixed permissions, which may not be sufficient to address dynamic security threats. Context-aware access control improves this approach by considering not only the user's identity but also additional contextual factors such as location, device type, time of access, and network conditions. By designing and applying access control systems that incorporate these contextual attributes, organizations can make more intelligent and flexible access decisions. This ensures that authorized users can access resources only under appropriate conditions, thereby strengthening security, reducing the risk of unauthorized access, and supporting more adaptive cybersecurity practices.

ABSTRACT

In the modern digital world, protecting sensitive data and systems from unauthorized access is a major concern for organizations. Traditional access control mechanisms mainly rely on user roles or static permissions, which may not be sufficient to address evolving security threats. Context-aware access control enhances security by incorporating additional contextual factors along with user identity when making access decisions. These factors may include location, device type, time of access, and network conditions. By designing and applying context-aware access control based on user identity, systems can dynamically evaluate access requests and enforce more flexible security policies. This approach ensures that users are granted access only under appropriate conditions, thereby reducing the risk of security breaches and improving overall system protection. The implementation of such adaptive access control mechanisms plays a vital role in strengthening modern cybersecurity frameworks and ensuring secure access to organizational resources.

With the rapid growth of digital technologies and online services, ensuring secure access to information systems has become a critical challenge. Traditional access control systems mainly depend on fixed roles and permissions, which may not effectively handle dynamic security risks. Context-aware access control provides a more advanced solution by combining user identity with contextual information such as device type, location, time, and network environment. By designing and implementing such systems, organizations can make smarter access decisions and enforce security policies that adapt to different situations. This approach improves data protection, prevents unauthorized access, and enhances the overall security of information systems.

USE CASE SCENARIOS

Scenario 1: Finance Department Access to ERP System

A senior accountant in the finance department needs to access the organization's ERP system to complete monthly payroll tasks. The user connects remotely using a company-issued laptop and opens the ZPA Client Connector. The system authenticates the user through Azure Active Directory with multi-factor authentication (MFA). ZPA then evaluates the user's identity, verifies membership in the Finance ERP Users group, checks the device's security status such as active antivirus and disk encryption, and analyses the connection context. Since all security conditions are satisfied, the policy engine authorizes the request and establishes a secure micro-tunnel directly to the ERP application. The user is able to work within the ERP system but cannot access or view any other internal resources like development servers or HR databases.

Output:

• User authenticated and authorized through identity verification and group membership

• Secure micro-tunnel created only for ERP system access

• Other internal systems remain hidden and inaccessible

Scenario 2: Contractor Access to Project Repository

An external contractor is hired for a short-term software project and requires access only to a specific Git repository located on an internal server. The contractor is registered as a guest user in the organization's identity provider and added to the Project Contractors group. When the contractor logs in using the ZPA client, the system verifies their identity and checks device security requirements such as operating system updates and firewall status. Based on the security policy, ZPA allows access only to the Git repository through SSH or HTTPS during working hours between 9 AM and 5 PM.

Output:

- Time-restricted and role-based access control applied
- Contractor permitted to access only the Git repository
- No connectivity to other internal systems

Scenario 3: Executive Access to Sensitive Documents

A company vice president needs to review confidential merger and acquisition documents stored in an internal SharePoint site. While traveling abroad, the executive connects from a hotel business center using a company-managed tablet. After launching the ZPA Client Connector, the user authenticates through Azure Active Directory with MFA. ZPA evaluates the executive's identity, verifies group membership, and checks device compliance. However, the login location from a foreign country increases the risk level. Although access is permitted, the policy engine enforces additional restrictions. The executive is given read-only access to the SharePoint documents, while downloading and printing features are disabled. The system also monitors the session for suspicious activity and notifies the security team.

Output:

- Successful authentication with MFA and compliant device
- Read-only access granted with download and print restrictions
- Security team alerted due to unusual login location

Scenario 4: Remote Support Technician Access

An IT support technician needs to use internal tools such as ServiceNow and a remote desktop application to help employees with technical issues. The technician connects from a home office using a company-managed desktop computer and launches the ZPA Client Connector. After authenticating through Azure Active Directory with MFA, ZPA checks the technician's identity, confirms membership in the IT Support group, verifies device compliance.

Output:

- User authenticated and authorized in a trusted environment
- Secure access granted to ServiceNow and remote desktop tools
- Least-privilege access maintained

Scenario 5: High-Risk Application Access from Public Wi-Fi

A marketing employee needs to update campaign details in the Salesforce system while working from a coffee shop using a company smartphone. After launching the ZPA Client Connector, the user authenticates through Azure Active Directory with MFA. ZPA checks the user's identity, group membership in Marketing Salesforce Users, and device compliance. However, the system identifies the public Wi-Fi network as a high-risk environment. Instead of blocking access completely, the policy engine routes the session through a secure browser isolation service. This service runs the Salesforce session in a temporary secure environment, preventing sensitive data from being stored on the device or exposed to the public network.

Output:

- User authenticated with compliant device but risky network detected
- Access provided through secure browser isolation
- Data protection enforced with restrictions on copying, downloading, and local storage

STAGE – 1

Title

Design and Implementation of Context-Aware Access Controls Based on User Identity, Device Posture, Location, and Application Risk

Overview

As cyber threats continue to evolve and the traditional network perimeter becomes less relevant, many organizations are moving away from legacy Virtual Private Network (VPN)–based remote access solutions. Conventional VPN systems typically grant users broad access to internal networks once they are authenticated. This approach increases the organization's attack surface and creates opportunities for attackers to move laterally within the network if a user account or device is compromised.

To address these security challenges, organizations are adopting the **Zero Trust security model**, which is based on the principle of **"never trust, always verify."** Instead of relying solely on network location to grant access, Zero Trust continuously verifies users, devices, and other contextual factors before allowing access to corporate resources.

This project focuses on implementing a **Zero Trust Access solution using Zscaler Private Access (ZPA)**. ZPA is a cloud-based service that provides secure access to internal applications without exposing the organization's network to the internet. It enables identity-driven, application-specific connectivity between users and the resources they need.

Unlike traditional VPNs that connect users directly to the corporate network, ZPA uses a **service-initiated connection model** to create a secure, one-to-one connection between an authenticated user and a specific application. This ensures that users can only access the applications for which they are authorized. Access decisions are made based on multiple contextual factors such as user identity, device posture, geographic location, and the risk level of the application.

By implementing ZPA, the organization's internal network and IP address space remain hidden from the public internet. This significantly reduces the risk of common cyberattacks such as port scanning, unauthorized access attempts, and distributed denial-of-service (DDoS) attacks, since the applications are not directly exposed.

In addition, this project enforces a least-privilege access model, ensuring that users and devices are granted only the level of access necessary to perform their tasks. Because users connect only to specific applications rather than the entire network, they cannot see or access other internal systems. This approach effectively prevents lateral movement within the network.

Overall, the adoption of Zero Trust architecture through ZPA improves the organization's security posture by minimizing the attack surface, strengthening access control mechanisms, and simplifying remote access management. This modern approach enables organizations to provide secure and efficient access to applications while maintaining strong protection against emerging cybersecurity threats.

LIST OF VULNERABILITY:

S.NO	VULNERABILITY NAME	CWE NO
1	Relative Path Traversal	23
2	Configuration	16
3	Use of obsolete function	447
4	Insufficient Entropy	331
5	Improper Input validation	20
6	Trust Boundary violation	501
7	Session Fixation	384
8	Unstructured Search Path	426
9	Insufficient logging	778
10	Divided by zero	369

Report

1. **Vulnerability Name : Relative Path Traversal**

   CWE-23

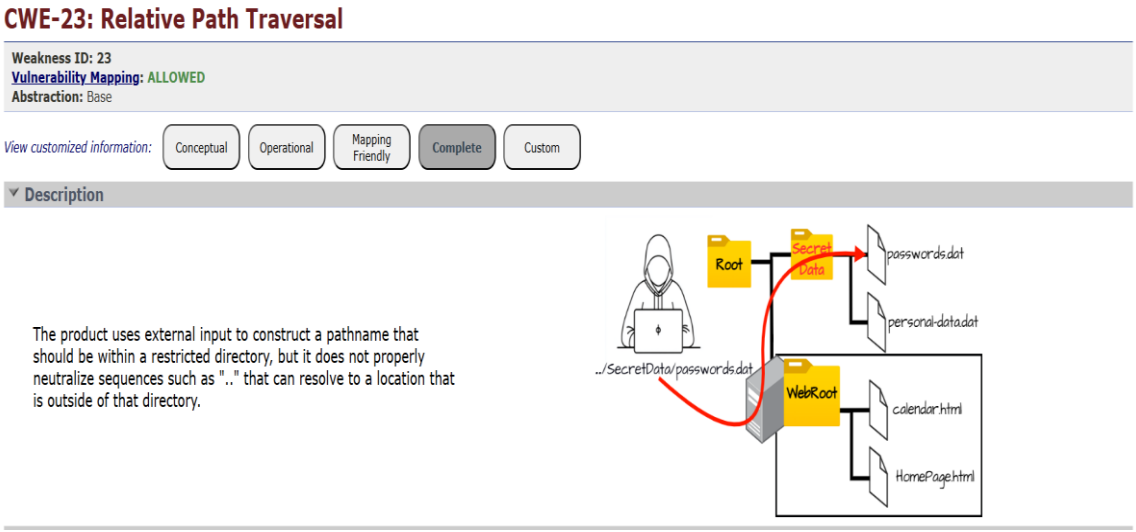
   OWASP Category A05:2021 – Security Misconfiguration

   Description

   Relative Path Traversal occurs when a web application does not properly validate user-supplied file paths. Attackers can manipulate the path using sequences such as ../ to move outside the intended directory and access restricted files on the server.

   Business Impact

   If exploited, attackers may gain access to confidential files such as passwords, personal data, or configuration files. This can lead to information disclosure and potential compromise of the system.



Vulnerability path:

http://testfire.net/showfile.jsp?file=../../SecretData/passwords.dat

Steps to Reproduce

- Access the URL

2. Vulnerability Name

Improper Configuration / Security Misconfiguration

CWE ID: CWE-16
OWASP Category: A02:2025 – Security Misconfiguration

Description

Security Misconfiguration occurs when a system, application, or service is deployed with insecure default settings, incomplete configurations, or unnecessary features enabled. Attackers can exploit these misconfigurations to gain unauthorized access, escalate privileges, or access sensitive data.

Business Impact

Exploitation of configuration weaknesses can lead to:

- Unauthorized access to confidential data.
- Service disruption (DoS attacks).
- Privilege escalation.
- Exposure of internal network details.

This can compromise system integrity, confidentiality, and availability, and potentially lead to regulatory or reputational consequences.

CWE CATEGORY: Configuration

Category ID: 16
Vulnerability Mapping: PROHIBITED

Summary

Weaknesses in this category are typically introduced during the configuration of the software.

Membership

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	933	OWASP Top Ten 2013 Category A5 - Security Misconfiguration
MemberOf	C	1032	OWASP Top Ten 2017 Category A6 - Security Misconfiguration
MemberOf	C	1349	OWASP Top Ten 2021 Category A05:2021 - Security Misconfiguration
MemberOf	C	1437	OWASP Top Ten 2025 Category A02:2025 - Security Misconfiguration

Vulnerability Mapping Notes

Usage: PROHIBITED *(this CWE ID must not be used to map to real-world vulnerabilities)*

Reason: Category

Rationale:
This entry is a Category. Using categories for mapping has been discouraged since 2019. Categories are informal organizational groupings of weaknesses that can help CWE users with data aggregation, navigation, and browsing. However, they are not weaknesses in themselves [REF-1287]. This CWE entry may have become widely-used because of NIST's usage in NVD from 2008 to 2016 (see CWE-635 view, updated to the CWE-1003 view in 2016). Mapping is also Prohibited because this entry's status is Obsolete.

Comments:
As of CWE 4.9, "Configuration" is beginning to be treated as an aspect of the SDLC in which a product is directed (by a human or automated process) to perform an insecure behavior. CWE mapping should be conducted by analyzing the weakness in the behavior that has been set by the configuration, such as those related to access control (descendants of CWE-284) or resource management (CWE-400), etc.

Vulnerable Path

Examples of misconfigurations:

1. Default admin credentials on a web application:
   http://example.com/admin -> Username: admin / Password: admin123

2. Publicly accessible S3 bucket:
   https://example-bucket.s3.amazonaws.com

Steps to Reproduce

1. Identify the system, application, or service to test.
2. Attempt to access default or common administration interfaces.
3. Test for default credentials (e.g., admin/admin, root/root).
4. Scan for exposed ports or services using tools like Nmap.
5. Check for unnecessary features enabled (e.g., debug pages, directory listings).
6. Document any access gained that should be restricted.

3. **Vulnerability Name- Use of Obsolete Function**

CWE ID: CWE-477 – Use of Obsolete Function
OWASP Category: A05:2021 – Security Misconfiguration

Description

This vulnerability occurs when an application relies on **deprecated or obsolete functions** in its code. Such functions may no longer receive security updates, may behave unpredictably, or may introduce weaknesses that attackers can exploit.

Common scenarios include:

- Using outdated cryptographic functions (e.g., MD5, SHA-1).
- Calling obsolete APIs that are no longer supported or patched.
- Relying on legacy libraries with known vulnerabilities.

Business Impact

Exploitation or reliance on obsolete functions can lead to:

- Compromise of data integrity or confidentiality.
- Unexpected application behavior or crashes.
- Vulnerabilities introduced via known exploits against the deprecated function.
- Increased risk of system compromise due to unpatched weaknesses.

CWE-447: Unimplemented or Unsupported Feature in UI

Weakness ID: 447
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

A UI function for a security feature appears to be supported and gives feedback to the user that suggests that it is supported, but the underlying functionality is not implemented.

Common Consequences

Impact	Details
Varies by Context; Unexpected State	**Scope: Other**

Potential Mitigations

Phase(s)	Mitigation
Testing	Perform functionality testing before deploying the application.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf		446	UI Discrepancy for Security Feature
ChildOf		671	Lack of Administrator Control over Security

Vulnerable path

http://example.com/register

Steps to Reproduce

1. Review the application source code for function calls or APIs marked as **deprecated** in official documentation.

2. Identify any functions that are obsolete or replaced by more secure alternatives.
3. Verify that these functions are still actively executed during application runtime.
4. Confirm if using these functions can lead to security issues (e.g., weak hashing, insecure encryption, or unpatched APIs).

4.Vulnerability Name - Insufficient Entropy

CWE ID: CWE-331
OWASP Category: A03:2021 – Cryptographic Failures

Description

CWE-331 occurs when an application generates cryptographic keys, tokens, session IDs, or other secrets using **predictable or low-entropy random values**. Attackers can guess or brute-force these values due to insufficient randomness, leading to compromise of sensitive information or authentication mechanisms.

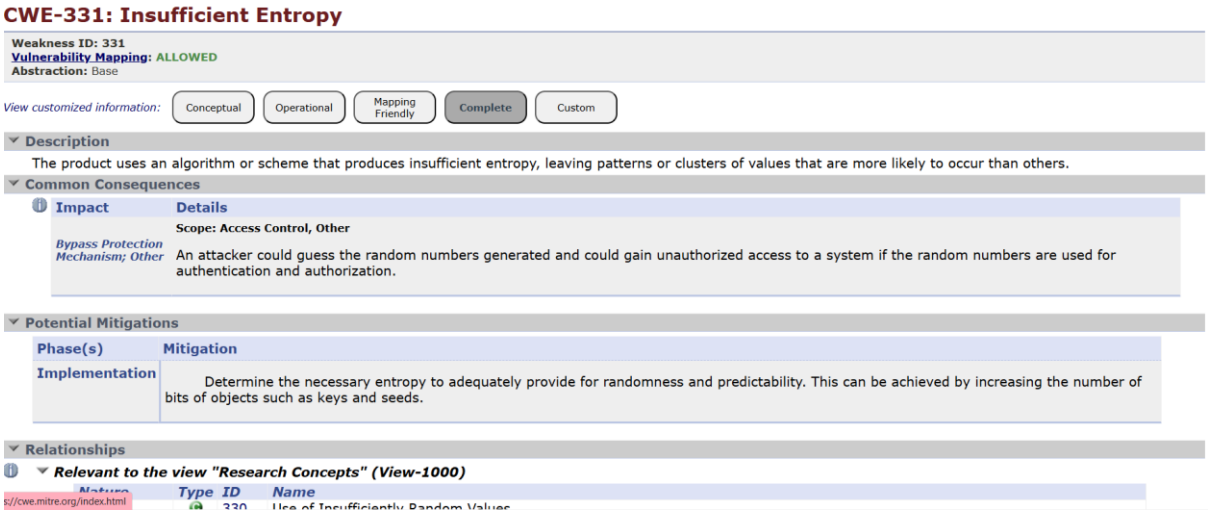
Common causes include:

- Using predictable random number generators (e.g., Math.random() in JavaScript, rand() in C).
- Reusing seeds for random number generation.
- Generating keys or tokens with low bit-length or poor algorithms.

Business Impact

Exploitation of this vulnerability may result in:

- Compromise of user accounts or authentication tokens.
- Predictable cryptographic keys enabling decryption of sensitive data.
- Unauthorized access to systems or services.
- Data breach and potential regulatory penalties.



CWE-331: Insufficient Entropy

Weakness ID: 331
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product uses an algorithm or scheme that produces insufficient entropy, leaving patterns or clusters of values that are more likely to occur than others.

Common Consequences

Impact	Details
Bypass Protection Mechanism; Other	Scope: Access Control, Other An attacker could guess the random numbers generated and could gain unauthorized access to a system if the random numbers are used for authentication and authorization.

Potential Mitigations

Phase(s)	Mitigation
Implementation	Determine the necessary entropy to adequately provide for randomness and predictability. This can be achieved by increasing the number of bits of objects such as keys and seeds.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
		330	Use of Insufficiently Random Values

Vulnerable Path

POST http://example.com/api/generateToken

Steps to Reproduce

1. Access the token generation endpoint:
2. Capture several generated tokens.
3. Observe that tokens follow a predictable pattern or limited numeric range.
4. Attempt brute-force using common numeric sequences to access other users' sessions.
5. Confirm that tokens can be guessed or predicted due to insufficient entropy.

5. Vulnerability Name – Improper Input Validation

CWE ID: CWE-20
OWASP Category: A03:2021 – Injection

Description

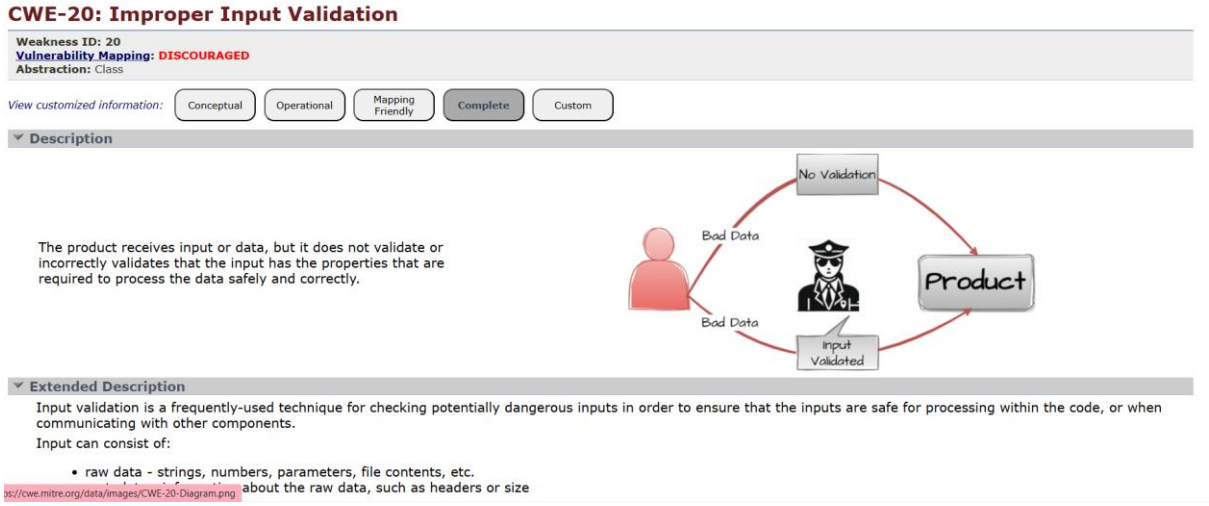
CWE-20 (Improper Input Validation) occurs when an application fails to properly validate or sanitize user-supplied input before processing it. When input validation is missing or insufficient, attackers can send malicious data that the application trusts and processes, potentially leading to various security issues such as injection attacks, unauthorized actions, application crashes, or data corruption.

Improper validation may include failing to verify input format, length, type, range, or allowed characters.

Business Impact

Exploitation of this vulnerability may result in:

- Injection attacks such as SQL Injection or Command Injection.
- Unauthorized modification or disclosure of sensitive data.
- Application crashes or denial of service.
- Execution of unintended application logic.



Vulnerable Path

POST http://example.com/api/updateProfile

Steps to Reproduce

1. Access the vulnerable endpoint:
3. Send the request using an interception tool (e.g., Burp Suite).
4. Observe that the application processes the input without proper validation.
5. Confirm that the application accepts malformed or malicious input, demonstrating insufficient input validation.

6. **Vulnerability Name – Trust Boundary Violation**

CWE ID: CWE-501
OWASP Category: A04:2021 – Insecure Design

Description

CWE-501 (Trust Boundary Violation) occurs when an application incorrectly assumes that data from an external or less-trusted source is safe and trustworthy without proper validation or verification. A trust boundary represents the point where data moves from one trust level to another, such as from a user to a server, from a client application to a backend API, or between internal services.

If an application fails to enforce validation or security checks when crossing this boundary, attackers can manipulate data to gain unauthorized access, escalate privileges, or bypass security controls.

Business Impact

Exploitation of this vulnerability may result in:

- Unauthorized access to sensitive resources or administrative functionality.
- Privilege escalation by modifying trusted parameters.
- Manipulation of system logic or user identity.
- Exposure or modification of confidential data.
- Potential compliance violations and reputational damage.

CWE-501: Trust Boundary Violation

Weakness ID: 501
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product mixes trusted and untrusted data in the same data structure or structured message.

Extended Description

A trust boundary can be thought of as line drawn through a program. On one side of the line, data is untrusted. On the other side of the line, data is assumed to be trustworthy. The purpose of validation logic is to allow data to safely cross the trust boundary - to move from untrusted to trusted. A trust boundary violation occurs when a program blurs the line between what is trusted and what is untrusted. By combining trusted and untrusted data in the same data structure, it becomes easier for programmers to mistakenly trust unvalidated data.

Common Consequences

Impact	Details
Bypass Protection Mechanism	Scope: Access Control

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	P	664	Improper Control of a Resource Through its Lifetime
PeerOf	B	349	Acceptance of Extraneous Untrusted Data With Trusted Data

Relevant to the view "Software Development" (View-699)

Vulnerable Path

POST http://example.com/api/updateUserRole

Steps to Reproduce

1. Access the user role update endpoint:
2. Send the modified request to the server.
3. Observe that the application accepts the modified role without proper authorization checks.

7.Vulnerability Name – Session Fixation

CWE ID: CWE-384
OWASP Category: A07:2021 – Authentication Failures

Description

CWE-384 (Session Fixation) occurs when a web application does not generate a new session identifier after a user successfully authenticates. Instead, the application continues to use the existing session ID provided before login.

An attacker can exploit this vulnerability by forcing or tricking a victim into using a known session ID. Once the victim authenticates, the attacker can reuse the same session ID to gain unauthorized access to the victim's authenticated session.

This issue typically arises when session management mechanisms fail to invalidate or regenerate session identifiers during authentication or privilege changes.

Business Impact

Exploitation of this vulnerability may result in:

- Unauthorized access to authenticated user sessions.
- Account takeover without requiring user credentials.
- Exposure or modification of sensitive user data.
- Unauthorized actions performed on behalf of legitimate users.
- Loss of user trust and potential regulatory consequences.

CWE-384: Session Fixation

Weakness ID: 384 (Structure: Composite**)**
Vulnerability Mapping: ALLOWED

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

Authenticating a user, or otherwise establishing a new user session, without invalidating any existing session identifier gives an attacker the opportunity to steal authenticated sessions.

Extended Description

Such a scenario is commonly observed when:

- A web application authenticates a user without first invalidating the existing session, thereby continuing to use the session already associated with the user.
- An attacker is able to force a known session identifier on a user so that, once the user authenticates, the attacker has access to the authenticated session.
- The application or container uses predictable session identifiers.

In the generic exploit of session fixation vulnerabilities, an attacker creates a new session on a web application and records the associated session identifier. The attacker then causes the victim to associate, and possibly authenticate, against the server using that session identifier, giving the attacker access to the user's account through the active session.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	**Scope: Access Control**

Potential Mitigations

Phase(s)	Mitigation

tps://cwe.mitre.org/index.html

Vulnerable Path

POST http://example.com/login

Steps to Reproduce

1. Access the application and obtain a valid session ID before authentication.
2. Capture the session cookie (e.g., SESSIONID=123456) using a proxy tool such as **Burp Suite**.
3. Send the session ID to a victim user (e.g., via a crafted link or shared session cookie).
4. The victim logs into the application while using the attacker-provided session ID.
5. Observe that the application does **not regenerate a new session ID** after successful authentication.

8.Vulnerability Name – Untrusted Search Path

CWE ID: CWE-426
OWASP Category: A05:2021 – Software are data integrity Failure

Description

CWE-426 (Untrusted Search Path) occurs when an application executes external programs or loads resources without specifying their full absolute path. Instead, the system searches for the executable or resource using environment paths (such as the system PATH).

If an attacker can place a malicious executable with the same name in a directory that is searched before the legitimate location, the application may execute the attacker's program instead of the intended one. This can lead to arbitrary code execution with the privileges of the vulnerable application.

This vulnerability commonly appears in operating system calls, scripts, scheduled tasks, or services that rely on relative paths.

Business Impact

- Execution of malicious programs on the system.
- Privilege escalation if the application runs with elevated permissions.
- Compromise of system integrity and confidentiality.
- Unauthorized access to sensitive resources.
- Potential system takeover by attackers.
-

CWE-426: Untrusted Search Path

Weakness ID: 426
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

Alternate Terms

Untrusted Path

Common Consequences

Impact	Details
Gain Privileges or Assume Identity; Execute Unauthorized Code	**Scope: Integrity, Confidentiality, Availability, Access Control** There is the potential for arbitrary code execution with privileges of the vulnerable program

Vulnerable path

POST http://example.com/api/runBackup

Steps to Reproduce

1. Identify an application feature that executes a system command without using an absolute path.
2. Observe that the application relies on the system PATH variable to locate the script.

3. Create a malicious executable with the same name (backup.sh) in a directory that appears earlier in the search path.
4. Place the malicious script in a directory accessible to the application.
5. Observe that the application executes the malicious script instead of the legitimate one.

9. Vulnerability Name – Insufficient Logging

CWE ID: CWE-778
OWASP Category: A09:2021 – Security Logging and Monitoring Failures

Description

CWE-778 (Insufficient Logging) occurs when an application fails to generate adequate logs for important security-related events. Without proper logging, malicious activities such as unauthorized access attempts, privilege escalation, or system misuse may go undetected.

Logging plays a critical role in detecting attacks, performing forensic analysis, and responding to incidents. When critical actions are not logged or logs lack sufficient details (such as timestamps, user IDs, or IP addresses), security teams may be unable to identify or investigate security breaches effectively

Business Impact

Exploitation of this vulnerability may result in:

- Inability to detect or investigate security incidents.
- Delayed response to malicious activities.
- Difficulty performing forensic analysis after a breach.
- Increased risk of prolonged unauthorized access.
- Non-compliance with regulatory and security standards.

CWE-778: Insufficient Logging

Weakness ID: 778
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

When a security-critical event occurs, the product either does not record the event or omits important details about the event when logging it.

Extended Description

When security-critical events are not logged properly, such as a failed login attempt, this can make malicious behavior more difficult to detect and may hinder forensic analysis after an attack succeeds.

As organizations adopt cloud storage resources, these technologies often require configuration changes to enable detailed logging information, since detailed logging can incur additional costs. This could lead to telemetry gaps in critical audit logs. For example, in Azure, the default value for logging is disabled.

Common Consequences

Impact	Details
Hide Activities	**Scope: Non-Repudiation** If security critical information is not recorded, there will be no trail for forensic analysis and discovering the cause of problems or the source of attacks may become more difficult or impossible.

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design	Use a centralized logging mechanism that supports multiple levels of detail.

Vulnerable Path

POST http://example.com/api/login

Steps to Reproduce

1. Attempt multiple failed login attempts using incorrect credentials.
2. Verify whether the application generates logs for these failed authentication attempts.
3. Observe that the application does not record details such as:
4. Attempt additional actions such as password changes or access to restricted resources.

10. Vulnerability Name – Divide by Zero

CWE ID: CWE-369
OWASP Category: A05:2021 – Security Misconfiguration (or A06:2021 – Vulnerable and Outdated Components depending on context)

Description

CWE-369 (Divide by Zero) occurs when an application performs a division operation without verifying whether the denominator is zero. If a zero value is used in a division calculation, it can cause runtime errors, application crashes, or unexpected behavior.

Attackers may intentionally provide input that results in a zero denominator, triggering application failures. In some cases, this may lead to denial-of-service (DoS) conditions or expose internal system information through error messages.

Business Impact

- Application crashes or service disruption.
- Denial-of-Service (DoS) conditions.
- Exposure of internal error messages or stack traces.
- Reduced system reliability and availability.
- Potential exploitation of further vulnerabilities through error information leakage.

CWE-369: Divide By Zero

Weakness ID: 369
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product divides a value by zero.

Extended Description

This weakness typically occurs when an unexpected value is provided to the product, or if an error occurs that is not properly detected. It frequently occurs in calculations involving physical dimensions such as size, length, width, and height.

Common Consequences

Impact	Details
DoS: Crash, Exit, or Restart	**Scope: Availability** A Divide by Zero results in a crash.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	P	682	Incorrect Calculation

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	189	Numeric Errors

Vulnerable Path

POST http://example.com/api/calculateDiscount

Steps to Reproduce

1. Access the calculation endpoint:
2. Intercept the request using a proxy tool (e.g., Burp Suite).
3. Modify the payload to set the denominator value to **0**.
4. Send the modified request to the server.
5. Observe that the application attempts to perform a division operation using **items count** as the denominator.